Exercício 10.1: Endurecimento com securityContext e PSA

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1. Por isso `enforce-version: v1.36`.
- Imagem `store-api:1.0` (a mesma dos capítulos anteriores). O Dockerfile dela define `USER 10001`, e o `runAsUser: 65532` do Pod sobrescreve esse valor.
- Sequência completa em `05-commands.sh`.

Manifestos

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: store-seguro
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: v1.36
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
---
apiVersion: v1
kind: Namespace
metadata:
  name: store-legado
  labels:
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/enforce-version: v1.36
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
  namespace: store-seguro
```

```yaml
spec:
  replicas: 2
  selector:
    matchLabels:
      app: store-api
  template:
    metadata:
      labels:
        app: store-api
    spec:
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        runAsGroup: 65532
        fsGroup: 65532
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: store-api
          image: store-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          readinessProbe:
            httpGet:
              path: /ready
              port: http
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              memory: 256Mi
          volumeMounts:
            - name: tmp
              mountPath: /tmp
      volumes:
        - name: tmp
          emptyDir: {}
```

Além do mínimo pedido, o Deployment tem `runAsGroup: 65532` (sem ele, o GID primário seria o da imagem) e `automountServiceAccountToken: false`, porque a `store-api` não fala com a API do Kubernetes. Nenhum dos dois é exigido pelo `restricted`.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-privilegiado
spec:
  hostNetwork: true
  hostPID: true
  containers:
    - name: root
      image: busybox:1.37
      command: ["sleep", "3600"]
      securityContext:
        privileged: true
        runAsUser: 0
      volumeMounts:
        - name: host
          mountPath: /host
  volumes:
    - name: host
      hostPath:
        path: /
```

O Pod privilegiado acumula de propósito tudo o que um atacante gostaria de ter: `privileged`, root, `hostNetwork`, `hostPID` e o `/` do nó montado via `hostPath`.

Namespace e Deployment

```
$ kubectl get ns store-seguro --show-labels
NAME           STATUS   AGE   LABELS
store-seguro   Active   10s   kubernetes.io/metadata.name=store-seguro,pod-
security.kubernetes.io/audit=restricted,pod-security.kubernetes.io/enforce-version=v1.36,pod-
security.kubernetes.io/enforce=restricted,pod-security.kubernetes.io/warn=restricted

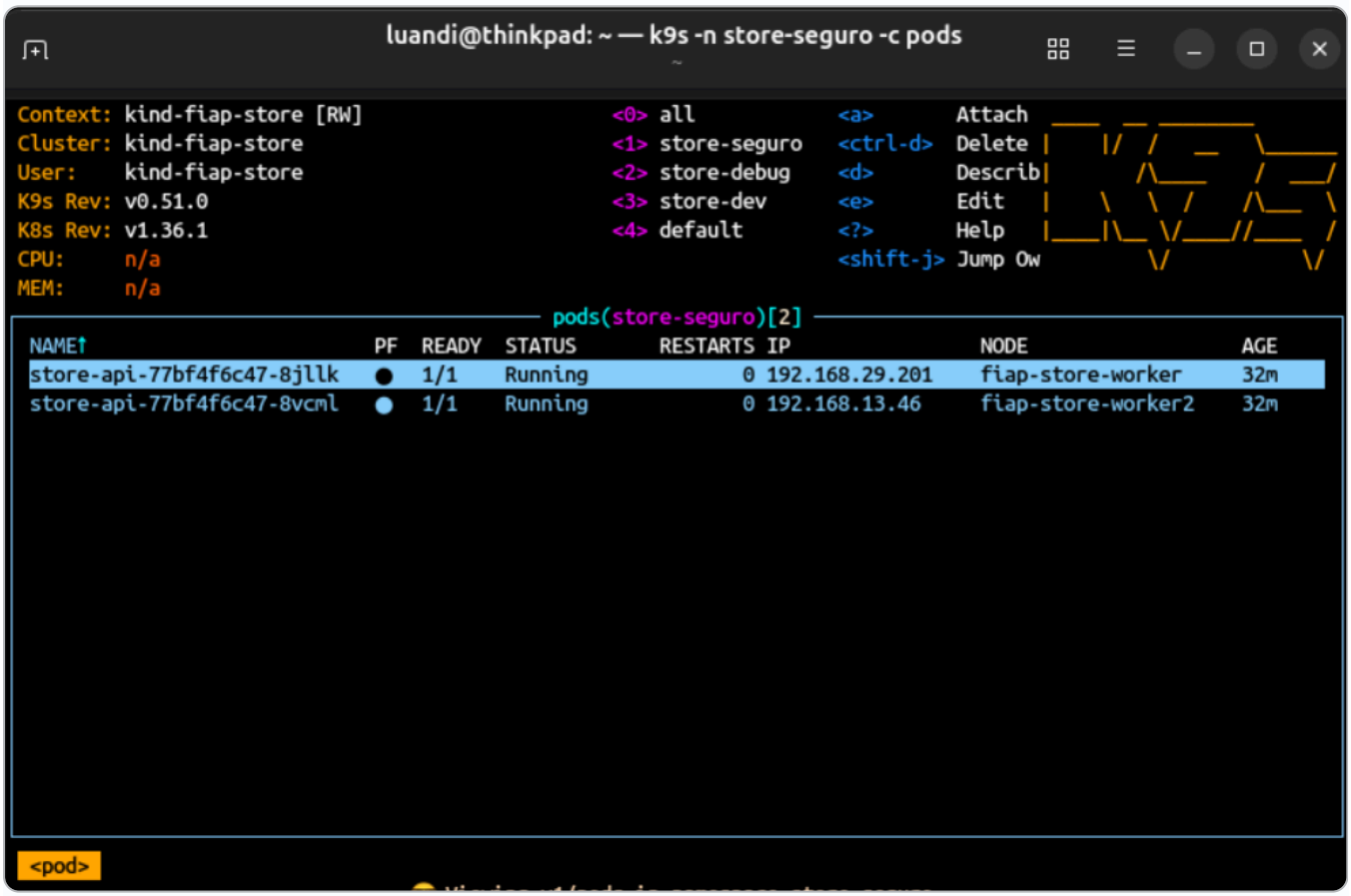
$ kubectl get pods -n store-seguro -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP               NODE
store-api-77bf4f6c47-8jllk   1/1     Running   0          11s   192.168.29.201   fiap-store-worker
store-api-77bf4f6c47-8vcml   1/1     Running   0          11s   192.168.13.46    fiap-store-worker2

$ kubectl exec -n store-seguro deploy/store-api -- id
uid=65532 gid=65532 groups=65532
```

Para conferir se o endurecimento está valendo de fato, e não só no YAML:

```
$ kubectl exec -n store-seguro deploy/store-api -- sh -c 'touch /app/x; touch /tmp/x && echo tmp-gravavel; grep Cap /proc/1/status'
touch: cannot touch '/app/x': Read-only file system
tmp-gravavel
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: 0000000000000000
CapAmb: 0000000000000000
```

Raiz somente-leitura, `/tmp` gravável pelo `emptyDir` e todas as capabilities zeradas, inclusive o conjunto *bounding*, o que impede ganhar capability depois.



Pod privilegiado em `store-seguro` (restricted)

```
$ kubectl apply -f 03-pod-privilegiado.yaml -n store-seguro
Error from server (Forbidden): error when creating "03-pod-privilegiado.yaml": pods "pod-
privilegiado" is forbidden:
violates PodSecurity "restricted:v1.36":
  host namespaces (hostNetwork=true, hostPID=true),
  privileged (container "root" must not set securityContext.privileged=true),
  allowPrivilegeEscalation ≠ false (container "root" must set
securityContext.allowPrivilegeEscalation=false),
  unrestricted capabilities (container "root" must set securityContext.capabilities.drop=["ALL"]),
  restricted volume types (volume "host" uses restricted volume type "hostPath"),
  runAsNonRoot ≠ true (pod or container "root" must set securityContext.runAsNonRoot=true),
  runAsUser=0 (container "root" must not set runAsUser=0),
  seccompProfile (pod or container "root" must set securityContext.seccompProfile.type to
"RuntimeDefault" or "Localhost")
```

8 violações listadas numa única mensagem (quebra de linha minha, para leitura).

Pod privilegiado em `store-legado` (baseline)

```
$ kubectl apply -f 03-pod-privilegiado.yaml -n store-legado
Error from server (Forbidden): error when creating "03-pod-privilegiado.yaml": pods "pod-
privilegiado" is forbidden:
violates PodSecurity "baseline:v1.36":
  host namespaces (hostNetwork=true, hostPID=true),
  hostPath volumes (volume "host"),
  privileged (container "root" must not set securityContext.privileged=true)
```

Também rejeitado, mas com **3 violações** em vez de 8. Para deixar a diferença entre os perfis explícita, um segundo Pod roda como root comum, sem nada de privilegiado:

```yaml
# root comum, sem privileged/host*: mostra onde baseline e restricted divergem
apiVersion: v1
kind: Pod
metadata:
  name: pod-root-simples
spec:
  containers:
    - name: root
      image: busybox:1.37
      command: ["sleep", "3600"]
```

```
$ kubectl apply -f 04-pod-root-simples.yaml -n store-legado
pod/pod-root-simples created
$ kubectl -n store-legado exec pod-root-simples -- id
uid=0(root) gid=0(root) groups=0(root),10(wheel)

$ kubectl apply -f 04-pod-root-simples.yaml -n store-seguro
Error from server (Forbidden): ... violates PodSecurity "restricted:v1.36": allowPrivilegeEscalation
≠ false (...),
unrestricted capabilities (...), runAsNonRoot ≠ true (...), seccompProfile (...)
```

Comparação

O `baseline` e o `restricted` respondem a perguntas diferentes. O `baseline` bloqueia **escalada conhecida para o nó**: `privileged`, namespaces do host (`hostNetwork`, `hostPID`) e `hostPath` são os três caminhos diretos de um container para o host, e por isso o Pod privilegiado foi rejeitado também no `store-legado`, só que por 3 motivos. Tudo o que não abre um desses caminhos passa: rodar como root, manter as capabilities padrão do runtime, não ter seccomp. É o que mostra o `pod-root-simples` aceito com `uid=0`. O `restricted` exige **boas práticas ativas**: além dos 3 itens do `baseline`, exige declarar não-root, `allowPrivilegeEscalation: false`, `drop: ["ALL"]` e seccomp, e restringe os tipos de volume. Por isso o mesmo Pod privilegiado recebe 8 violações, e até o root "inofensivo" é recusado. Na prática, `baseline` é um piso aceitável para namespaces legados que não conseguem migrar já, e `restricted` é o alvo de produção. A `store-api`, com o `securityContext` completo, roda nele sem nenhuma concessão.

Critérios de aceite

Critério	Resultado
2 Pods da `store-api` em `Running 1/1` no namespace `restricted`	ok
`id` retorna `uid=65532`	ok
Pod privilegiado rejeitado com `violates PodSecurity "restricted"` e pelo menos 4 violações	ok (8)
Mesmo Pod no `baseline` rejeitado ou aceito com justificativa	rejeitado (3 violações), mais o contraste do root comum aceito